```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_re
import matplotlib.pyplot as plt
import seaborn as sns
```

```
card_dataset = pd.read_csv('/content/creditcard.csv')
```

```
card_dataset.head()
```

	Time	V1	V2	V3	V4	V5	V6	V7	
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-

```
card_dataset.shape
```

```
(284807, 31)
```

```
card_dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
```

```
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

```
card_dataset.isnull().sum()
```

	0
Time	0
V1	0
V2	0
V3	0
V6	0
V7	0
V9	0
V10	0
V11	0
V16	0
V17	0
V19	0
V21	0
V23	0
V25	0
V27	0
V28	0
Amount	0

- 0 = Legit
- 1 = Fraud

```
card_dataset['Class'].value_counts()
```

Class	count
0	284315
1	492

dtype: int64

```
legit = card_dataset[card_dataset.Class == 0]
fraud = card_dataset[card_dataset.Class == 1]
```

```
print(legit.shape)
print(fraud.shape)
```

```
(284315, 31)
(492, 31)
```

```
legit.Amount.describe()
```

	Amount
count	284315.000000
mean	88.291022
std	250.105092
min	0.000000
25%	5.650000
50%	22.000000
75%	77.050000
max	25691.160000

dtype: float64

```
fraud.Amount.describe()
```

dtype: int64

	Amount
count	492.000000
mean	122.211321
std	256.683288
min	0.000000
25%	1.000000
50%	9.250000
75%	105.890000
max	2125.870000

dtype: float64

```
card_dataset.groupby('Class').mean()
```

Class	Time	V1	V2	V3	V4	V5	V6
0	94838.202258	0.008258	-0.006271	0.012171	-0.007860	0.005453	0.002419
1	80746.806911	-4.771948	3.623778	-7.033281	4.542029	-3.151225	-1.397737

Dataset is imbalance that's why I'm taking radom transactions from legit.

```
legit_sample = legit.sample(n=492)
```

```
credit_card_dataset = pd.concat([legit_sample, fraud], axis=0)
```

```
credit_card_dataset.head()
```

	Time	V1	V2	V3	V4	V5	V6	
137225	82072.0	-1.323993	0.077667	2.138717	0.557824	-1.177525	2.293411	-1
123646	77009.0	-0.936453	1.006224	0.830815	0.235568	-0.052606	-0.978703	0
50322	44442.0	-1.756421	2.470268	-3.696764	1.797293	-0.658082	-1.942159	-0
248436	153943.0	2.237654	-1.417598	-0.416335	-1.702766	-1.352776	-0.016486	-1
137825	82347.0	-1.151751	0.179532	1.793078	-0.411114	0.879027	-1.216259	0

```
credit_card_dataset.shape
```

```
(984, 31)
```

```
credit_card_dataset['Class'].value_counts()
```

Class	count
0	492
1	492

dtype: int64

```
credit_card_dataset.groupby('Class').mean()
```

Class	Time	V1	V2	V3	V4	V5	V6
0	93269.634146	0.106363	-0.025773	-0.004939	-0.017009	0.040600	-0.029021
1	80746.806911	-4.771948	3.623778	-7.033281	4.542029	-3.151225	-1.397737

```
X = credit_card_dataset.drop(columns='Class', axis=1)
Y = credit_card_dataset['Class']
```

```
print(X)
print(Y)
```

```
            Time        V1        V2  ...       V27       V28  Amount
137225   82072.0 -1.323993  0.077667  ... -1.008423 -0.698875   29.05
123646   77009.0 -0.936453  1.006224  ... -0.221643 -0.043787   15.73
50322    44442.0 -1.756421  2.470268  ...  0.145622 -0.199301   44.99
```

```
248436  153943.0  2.237654 -1.417598  ...  0.036971 -0.038817    4.95
137825   82347.0 -1.151751  0.179532  ... -0.241904 -0.170377   42.81
...          ...       ...       ...  ...       ...       ...     ...
279863  169142.0 -1.927883  1.125653  ...  0.292680  0.147968  390.00
280143  169347.0  1.378559  1.289381  ...  0.389152  0.186637    0.76
280149  169351.0 -0.676143  1.126366  ...  0.385107  0.194361   77.89
281144  169966.0 -3.113832  0.585864  ...  0.884876 -0.253700  245.00
281674  170348.0  1.991976  0.158476  ...  0.002988 -0.015309   42.53

[984 rows x 30 columns]
137225    0
123646    0
50322     0
248436    0
137825    0
         ..
279863    1
280143    1
280149    1
281144    1
281674    1
Name: Class, Length: 984, dtype: int64
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, strati
```

```
print(X.shape, X_train.shape, X_test.shape)
```

```
(984, 30) (787, 30) (197, 30)
```

```
model = LogisticRegression()
```

```
model.fit(X_train, Y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: C
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regressic
  n_iter_i = _check_optimize_result(
```

```
▾ LogisticRegression
LogisticRegression()
```

```
X_train_prediction = model.predict(X_train)
training_data_accuracy = accuracy_score(X_train_prediction, Y_train)
```

```
print('Accuracy on Training data : ', training_data_accuracy)
```

```
Accuracy on Training data :  0.9453621346886912
```

```
X_test_prediction = model.predict(X_test)
test_data_accuracy = accuracy_score(X_test_prediction, Y_test)
```

```
print('Accuracy score on Test Data : ', test_data_accuracy)
```

```
Accuracy score on Test Data :  0.9086294416243654
```

```
cm = confusion_matrix(Y_test, X_test_prediction)
print(cm)
```

```
[[94  5]
 [13 85]]
```

```
cp = classification_report(Y_test, X_test_prediction)
print(cp)
```

```
              precision    recall  f1-score   support

           0       0.88      0.95      0.91        99
           1       0.94      0.87      0.90        98

    accuracy                           0.91       197
   macro avg       0.91      0.91      0.91       197
weighted avg       0.91      0.91      0.91       197
```

```
plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Truth')
plt.title('Confusion Matrix')
plt.legend()
plt.show()
```